

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»

отчет по блоку задач №4 (Яндекс.Контест)

Выполнил:

Студент группы Р3231

Тимошкин Р.В.

Преподаватель:

Косяков М.С.

Санкт-Петербург, 2025г.

Задача №14 «N. Свинки-копилки»

Описание:

Есть n копилек, которые можно разбить или открыть ключом. Ключи лежат в некоторых из копилек. Нужно получить содержимое всех копилек. Какое минимальное количество копилек необходимо разбить?

Доказательство:

Построим функциональный граф на n вершинах, где из вершины i выходит ребро в a_i . В каждой компоненте такого графа есть ровно один цикл (все остальные вершины “пристёгнуты” к нему деревьями). Ключи для всех вершин вне циклов можно достать по направлению рёбер, а внутри каждого цикла нет внешнего ключа — значит, придётся разбить хотя бы одну копилку в каждом цикле. Итого ответ равен числу циклических компонент. Для подсчёта циклов достаточно обойти все вершины, отмечая посещения и фиксируя моменты, когда мы возвращаемся в уже отмеченную в рамках одного обхода вершину.

Алгоритмическая сложность:

- По времени - $O(n)$.
- По памяти: - $O(n)$.

Код:

```
1. #include <iostream>
2. #include <vector>
3.
4. namespace {
5.     auto CountCycles(int n, const std::vector<int>& banks) -> int {
6.         auto visited = std::vector<int>(n + 1, 0);
7.         auto cnt = 0;
8.
9.         for (auto i = 1; i <= n; ++i) {
10.             if (visited[i] != 0) {
11.                 continue;
12.             }
13.
14.             auto bank = i;
15.             while (visited[bank] == 0) {
16.                 visited[bank] = i;
17.                 bank = banks[bank];
18.             }
19.
20.             if (visited[bank] == i) {
21.                 ++cnt;
22.             }
23.         }
24.         return cnt;
25.     }
26. } // namespace
27.
28. auto main() -> int {
29.     auto count = 0;
30.     std::cin >> count;
31.
32.     auto banks = std::vector<int>(count + 1);
33.     for (auto i = 1; i <= count; ++i) {
34.         std::cin >> banks[i];
```

```

35.     }
36.
37.     auto result = CountCycles(count, banks);
38.     std::cout << result << "\n";
39.
40.     return 0;
41. }

```

Задача №15 «О. Долой списывание!»

Описание:

Есть пары лкшат, обменивающихся записками. Нужно определить, можно ли разделить их на две группы так, чтобы любой обмен записками осуществлялся от лкшонка одной группы лкшонку другой группы.

Доказательство:

Построим неориентированный граф с N вершинами и M рёбрами обменов. Требуется проверить, является ли этот граф двудольным: то есть можно ли раскрасить вершины в два цвета так, чтобы каждое ребро соединяло вершины разных цветов. Если граф двудольный, соответствующая раскраска даёт нужное разделение на группы; иначе найдётся ребро, оба конца которого получили одинаковый цвет, и разделить нельзя. Проверку двудольности выполняют с помощью BFS или DFS в $O(N+M)$.

Алгоритмическая сложность:

- По времени - $O(n+m)$.
- По памяти: - $O(n+m)$.

Код:

```

1. #include <stdint>
2. #include <iostream>
3. #include <queue>
4. #include <vector>
5.
6. namespace {
7. auto IsBipartite(int n, const std::vector<std::vector<int>>& adj) ->
    bool {
8.     std::vector<int> color(n + 1, -1);
9.
10.    for (auto start = 1; start <= n; ++start) {
11.        if (color[start] != -1) {
12.            continue;
13.        }
14.
15.        std::queue<int> queue;
16.        color[start] = 0;
17.        queue.push(start);
18.
19.        while (!queue.empty()) {
20.            auto next = queue.front();
21.            queue.pop();
22.
23.            for (auto elem : adj[next]) {
24.                if (color[elem] == -1) {
25.                    color[elem] =
static_cast<int>(static_cast<uint64_t>(color[next]) ^ 1U);
26.                    queue.push(elem);

```

```

27.         } else if (color[elem] == color[next]) {
28.             return false;
29.         }
30.     }
31. }
32. }
33. return true;
34. }
35. } // namespace
36.
37. auto main() -> int {
38.     auto count = 0;
39.     auto pair_count = 0;
40.     std::cin >> count >> pair_count;
41.
42.     std::vector<std::vector<int>> adj(count + 1);
43.     for (auto i = 0; i < pair_count; ++i) {
44.         auto first = 0;
45.         auto second = 0;
46.         std::cin >> first >> second;
47.
48.         adj[first].push_back(second);
49.         adj[second].push_back(first);
50.     }
51.
52.     std::cout << (IsBipartite(count, adj) ? "YES" : "NO") << "\n";
53.     return 0;
54. }

```